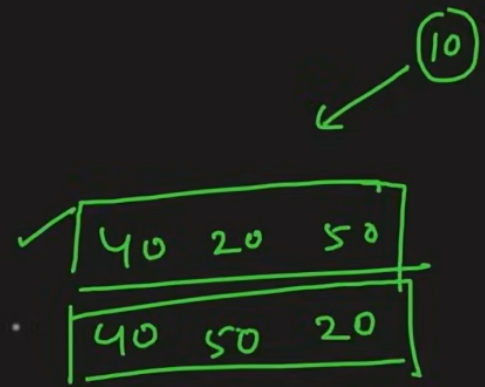


Construct a Binary Tree from PostOrder & Inorder.

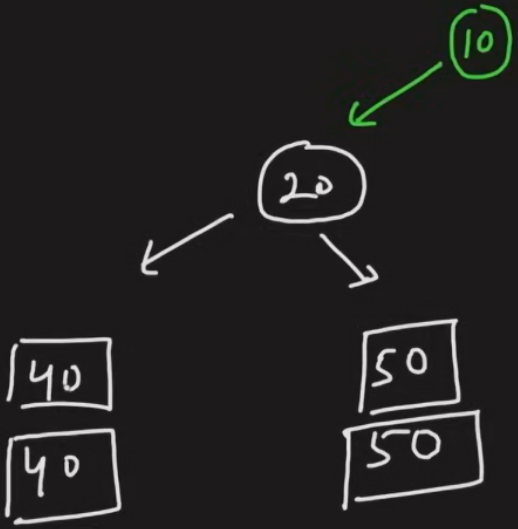
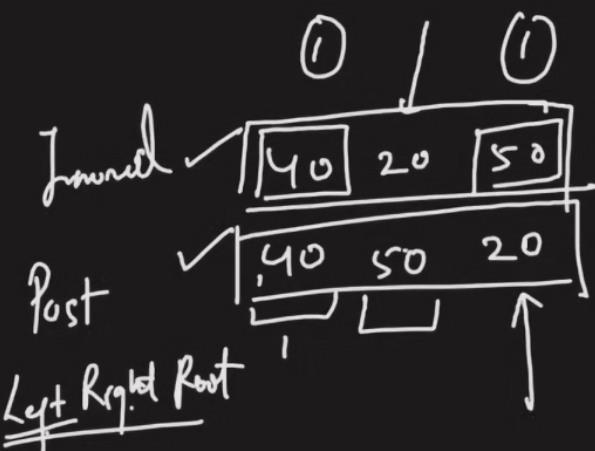
Inorder → [40 20 50 | 10 | 60 30]
(Left Root Right)



60 30
60 30

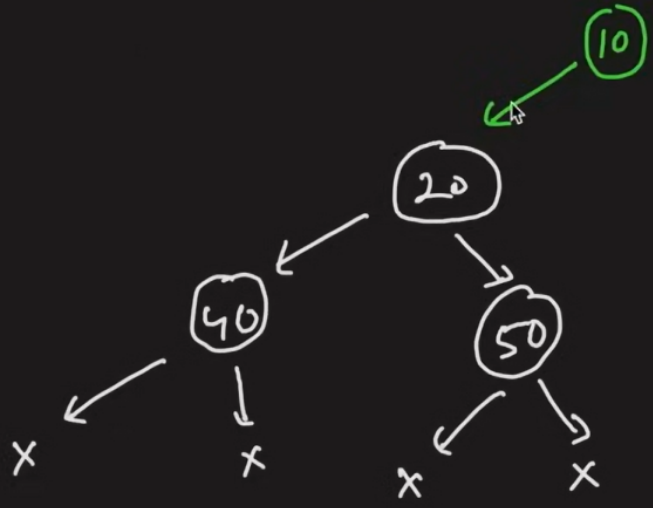
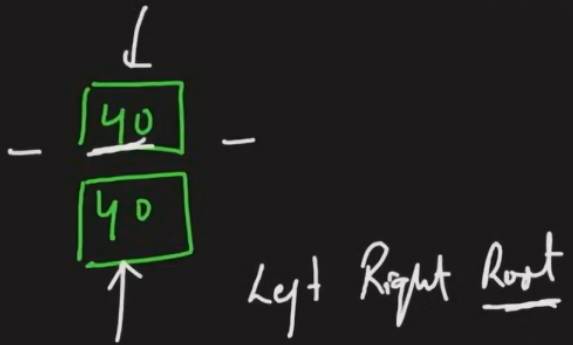
Postorder → [40 50 20] [60 30] 10
(Left Right Root)

Construct a Binary Tree from PostOrder & Inorder.



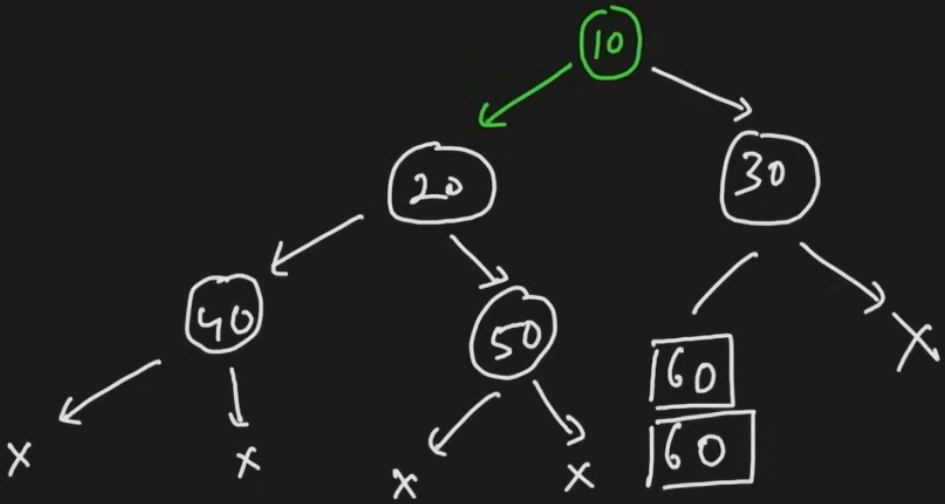
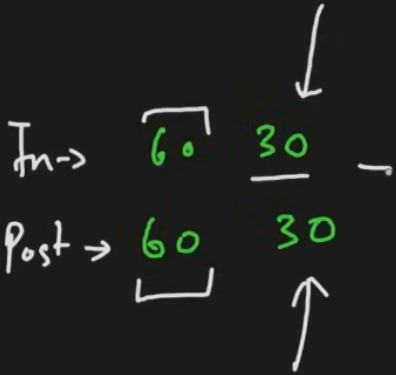
60 30
60 30

Construct a Binary Tree from PostOrder & Inorder.



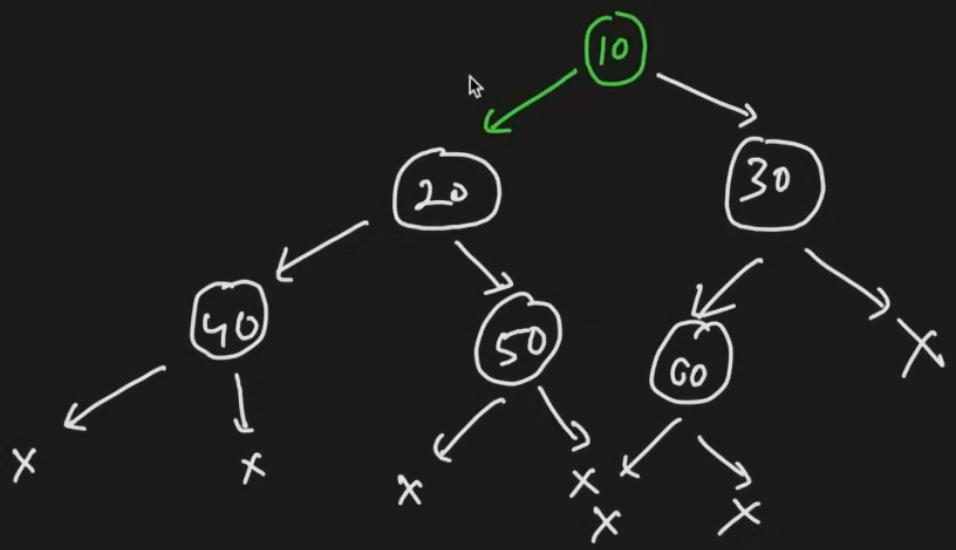
60 30
60 30

Construct a Binary Tree from PostOrder & Inorder.



Construct a Binary Tree from PostOrder & Inorder.

In → 60 -
Post → 60



```
2.30 C++
Autocomplete

1 class Solution {
2 public:
3     TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder) {
4         if (inorder.size() != postorder.size())
5             return NULL;
6         map<int,int> hm;
7         for (int i=0;i<inorder.size();++i)
8             hm[inorder[i]] = i;
9         return buildTreePostIn(inorder, 0, inorder.size()-1, postorder, 0,
10                                postorder.size()-1, hm);
11     }
12     TreeNode* buildTreePostIn(vector<int> &inorder, int is, int ie,
13                               vector<int> &postorder, int ps, int pe,
14                               map<int,int> &hm){
15         if (ps>pe || is>ie) return NULL;
16         TreeNode* root = new TreeNode(postorder[pe]);
17         int inRoot = hm[postorder[pe]];
18         int numsLeft = inRoot - is;
19         root->left = buildTreePostIn(inorder, is, inRoot-1,
20                                     postorder, ps, ps + numsLeft -1, hm);
21         root->right = buildTreePostIn(inorder, inRoot+1, ie,
22                                     postorder, ps + numsLeft, pe-1, hm);
23         return root;
24     }
25 };
26
27
28
29
30
31
32
```



```
Java
Autocomplete

1 class Solution {
2     public TreeNode buildTree(int[] inorder, int[] postorder) {
3         if (inorder == null || postorder == null || inorder.length != postorder.length)
4             return null;
5         HashMap<Integer, Integer> hm = new HashMap<Integer,Integer>();
6         for (int i=0;i<inorder.length;++i)
7             hm.put(inorder[i], i);
8         return buildTreePostIn(inorder, 0, inorder.length-1, postorder, 0,
9                                postorder.length-1, hm);
10    }
11    private TreeNode buildTreePostIn(int[] inorder, int is, int ie,
12                                     int[] postorder, int ps, int pe,
13                                     HashMap<Integer,Integer> hm){
14        if (ps>pe || is>ie) return null;
15        TreeNode root = new TreeNode(postorder[pe]);
16        int inRoot = hm.get(postorder[pe]);
17        int numsLeft = inRoot - is;
18        root.left = buildTreePostIn(inorder, is, inRoot-1,
19                                    postorder, ps, ps + numsLeft -1, hm);
20        root.right = buildTreePostIn(inorder, inRoot+1, ie,
21                                    postorder, ps + numsLeft, pe-1, hm);
22        return root;
23    }
24 }
25
26
27
28
29
30
31
32
```


L35. Construct the Binary Tree from Postorder and Inorder Traversal | C++ | Java

2.30
C++

Autocomplete

{} ~ ☰ ☲ ☳ ☴ ☵ ☶ ☷

```

1 class Solution {
2 public:
3     TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder) {
4         if (inorder.size() != postorder.size())
5             return NULL;
6
7         map<int,int> hm;
8
9         for (int i=0;i<inorder.size();++i)
10             hm[inorder[i]] = i;
11
12         return buildTreePostIn(inorder, 0, inorder.size()-1, postorder, 0,
13                                 postorder.size()-1, hm);
14     }
15     TreeNode* buildTreePostIn(vector<int> &inorder, int is, int ie,
16                               vector<int> &postorder, int ps, int pe,
17                               map<int,int> &hm){
18         if (ps>pe || is>ie) return NULL;
19
20         TreeNode* root = new TreeNode(postorder[pe]);
21
22         int inRoot = hm[postorder[pe]];
23         int numsLeft = inRoot - is;
24
25         root->left = buildTreePostIn(inorder, is, inRoot-1,
26                                     postorder, ps, ps + numsLeft - 1, hm);
27
28         root->right = buildTreePostIn(inorder, inRoot+1, ie,
29                                     postorder, ps + numsLeft, pe-1, hm);
30         return root;
31     }
32 };

```

10

